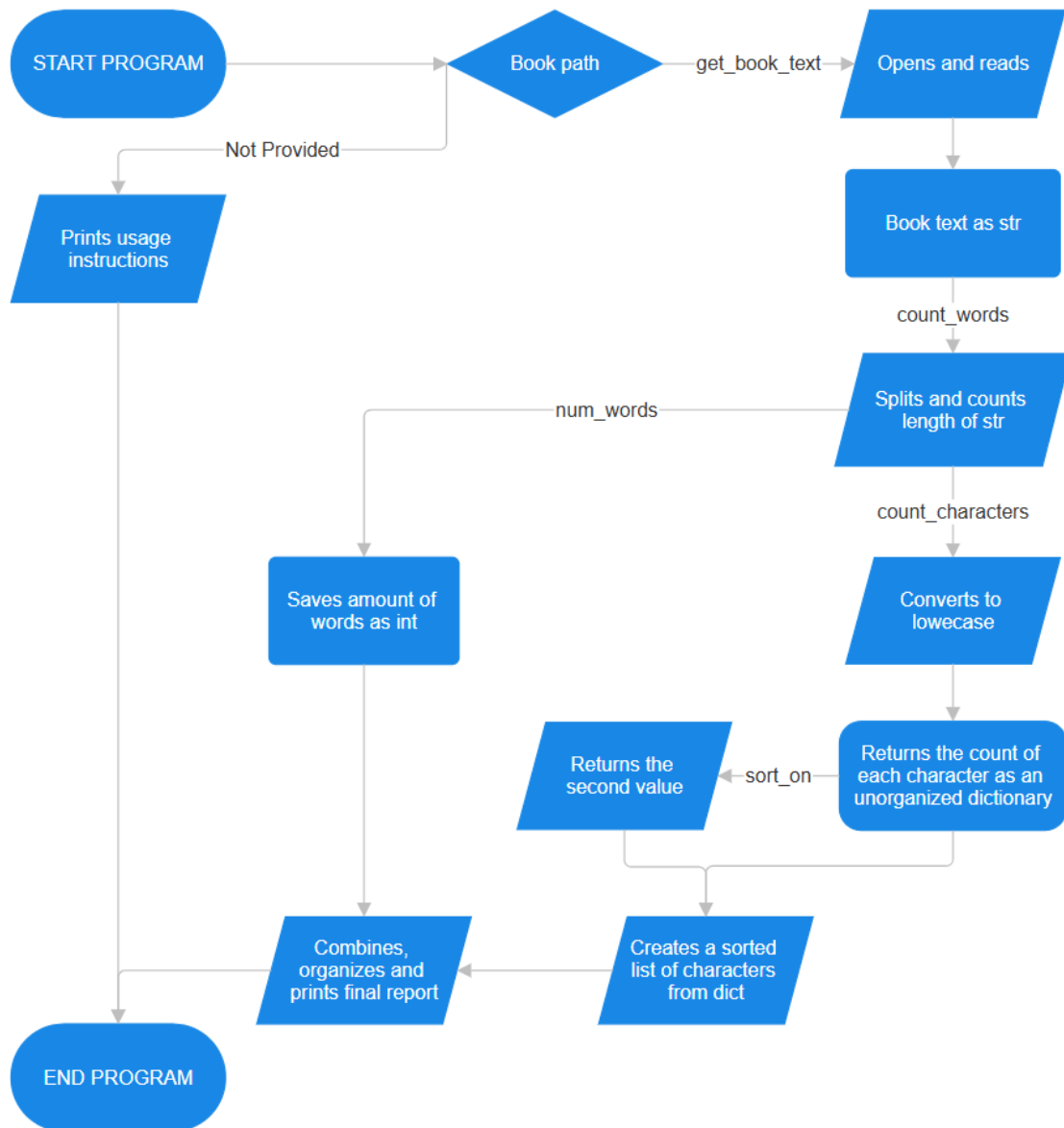


BookBot Project Notebook

IPO table for the program

Function	Input	Process	Output
<i>book_path</i>	<i>sys.argv[1]</i>	• Assigns <i>book_path</i> to the second <i>sys.argv</i>	• Saves <i>book_path</i> as <i>sys.argv[1]</i>
<i>get_book_text</i>	<i>path_to_file</i> (provided by user)	• Opens and Reads the book	• Returns Book as String called <i>text</i>
<i>count_words</i>	<i>text</i>	• Splits <i>text</i>	• Returns amount of Words as int called <i>num_words</i>
<i>count_characters</i>	<i>text</i>	• Converts each character to lowercase • Loops through the lowercase, adding counts for each character	• Returns character counts as Dictionary
<i>sort_on</i>	Accepts a Tuple	• Extracts the tuple's second value (int)	• Returns that value
<i>chars_dict_to_sorted_list</i>	Accepts Character Dictionary	• Loop over the dictionary's keys and adds it to a list	• Returns the sorted list of characters called <i>listOfSorted</i>
<i>print_report</i>	<i>book_path</i> , <i>num_words</i> , <i>listOfSorted</i>	• Combines organized data together • Loops over characters and removes non-alphabetic characters	• Prints final report

Data flow diagram



Design decisions and justifications

Why did I write the code the way I did? Firstly, separating the different code (main, stats) allowed for easier readability as well as allowing me to use pytest with ease. The main handles organising and printing, while the stats focus more on getting the actual statistics. Furthermore, in the test cases, I purposely tested how the system responded to unexpected data types - such as integers or empty inputs, or even non-English characters - preventing bugs and ensuring reliability when running the program. These design decisions ultimately enable my code to run smoothly for the user and allow for anyone to easily read and understand the logic behind the program.

Test cases with inputs and expected outputs

See inside “*test_stats.py*” on the git repository for all of the test cases with inputs and expected outputs.

Reflection on challenges and solutions

There were quite a few challenges faced in this BookBot project despite there being relatively little coding. One of the key challenges I faced was not understanding my own code - Initially I just gave functions random names based on what the Bootdev lesson was instructing me to do. This resulted in confusion and multiple incidents of mixing up my code. Finally, I decided to reorganize my code, renaming everything so I could understand what was doing what. Another problem that I faced was some built-in functions that I hadn't used before, such as “*sorted()*” or “*sys*” or “*.isalpha()*” functions. Luckily, Bootdev provides links to an explanation of the function, so I could just read through the description on how to use it. The pytest tests were also a bit of a fun challenge, encouraging me to go back to my code and try to understand how it would react to unusual inputs. Trying to work out why things like “[('a', 2), ('b', 2)]” fails required me to research my own code and see how it behaves. My program orders things from largest to smallest, I tried seeing what would happen if the value was the same. Turns out, it retains the insertion order when the values are the same!